Student Marks Matrix Analysis

1. Problem Statement

Write a Python program to analyze student marks stored in a matrix using nested lists. The program should calculate the total marks obtained by each student, calculate the average marks for each subject, and identify the top-performing student based on the highest total marks.

The program uses nested lists, nested loops, arithmetic operations, and conditional statements to process and analyze student marks.

2. Objectives

1. To store student marks in a two-dimensional matrix using nested lists.
2. To calculate the total marks obtained by each student.
3. To calculate the average marks in each subject.
4. To identify the top-performing student based on total marks.
5. To understand the use of nested loops and conditional statements in Python.

3. Assumptions

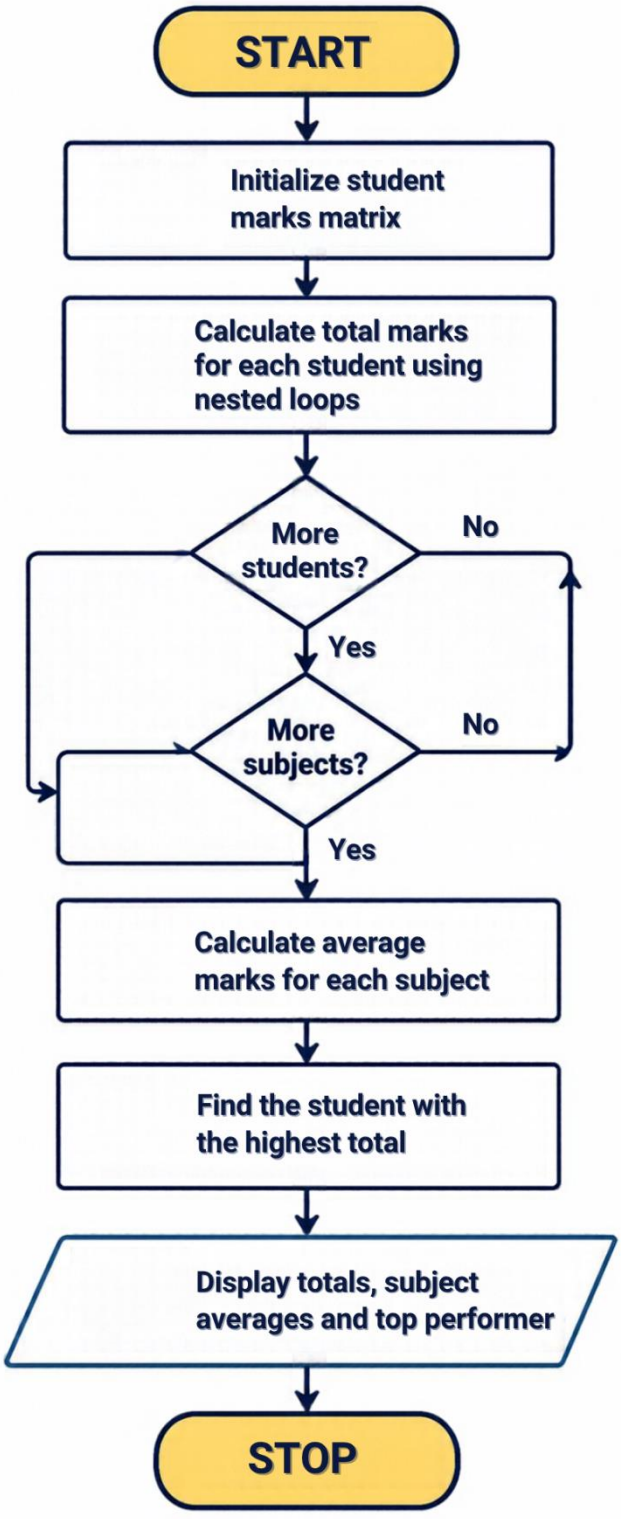
1. The program considers 4 students and 3 subjects.
2. Marks are entered as integers between 0 and 100.
3. Each row in the matrix represents one student.
4. Each column represents one subject.
5. All students have marks for all subjects.
6. If two students have the same highest total, the program identifies the first student with that total as the top performer.

4. Algorithm

1. Start the program.
2. Initialize a matrix containing marks of 4 students in 3 subjects.

3. Display the marks matrix.
4. For each student, initialize the total to zero.
5. Use a nested loop to add the marks of all subjects for each student.
6. Display the total marks of every student.
7. For each subject, add the marks of all students.
8. Calculate the average marks by dividing the subject total by the number of students.
9. Display the average marks for each subject.
10. Find the student with the highest total marks.
11. Display the top-performing student's number and total marks.
12. Stop the program.

5. Flowchart


START
Initialize student marks matrix
Calculate total marks for each student using nested loops
More students?
No
Yes
More subjects?
No
Yes
Calculate average marks for each subject
Find the student with the highest total
Display totals, subject averages and top performer
STOP

6. Pseudocode

```
BEGIN

   marks = [
      [85, 90, 78],
      [92, 88, 95],
      [76, 85, 80],
      [89, 91, 84]
   ]

   number_of_students = number of rows
   number_of_subjects = number of columns

   FOR each student i
      total = 0

      FOR each subject j
         total = total + marks[i][j]
      END FOR

      store total in student_totals
      display student total
   END FOR
```

```
    FOR each subject j
        subject_total = 0

        FOR each student i
            subject_total = subject_total + marks[i][j]
        END FOR

        average = subject_total / number_of_students
        display subject average
    END FOR

    highest_total = maximum student total
    top_student = index of highest total

    display top_student and highest_total

END
```

7. Python Actual Code

```
# Student Marks Matrix Analysis

# Marks of 4 students in 3 subjects
marks = [
    [85, 90, 78],
    [92, 88, 95],
```

```
    [76, 85, 80],
    [89, 91, 84]
]

number_of_students = len(marks)
number_of_subjects = len(marks[0])

# Calculate total marks of each student
student_totals = []

print("STUDENT TOTAL MARKS")

for i in range(number_of_students):
    total = 0

    for j in range(number_of_subjects):
        total += marks[i][j]

    student_totals.append(total)
    print("Student", i + 1, ":", total)

# Calculate average marks of each subject
print("\nSUBJECT AVERAGES")
```

```
for j in range(number_of_subjects):
    subject_total = 0

    for i in range(number_of_students):
        subject_total += marks[i][j]

    average = subject_total / number_of_students
    print("Subject", j + 1, ":", round(average, 2))

# Identify the top-performing student
highest_total = max(student_totals)
top_student = student_totals.index(highest_total) + 1

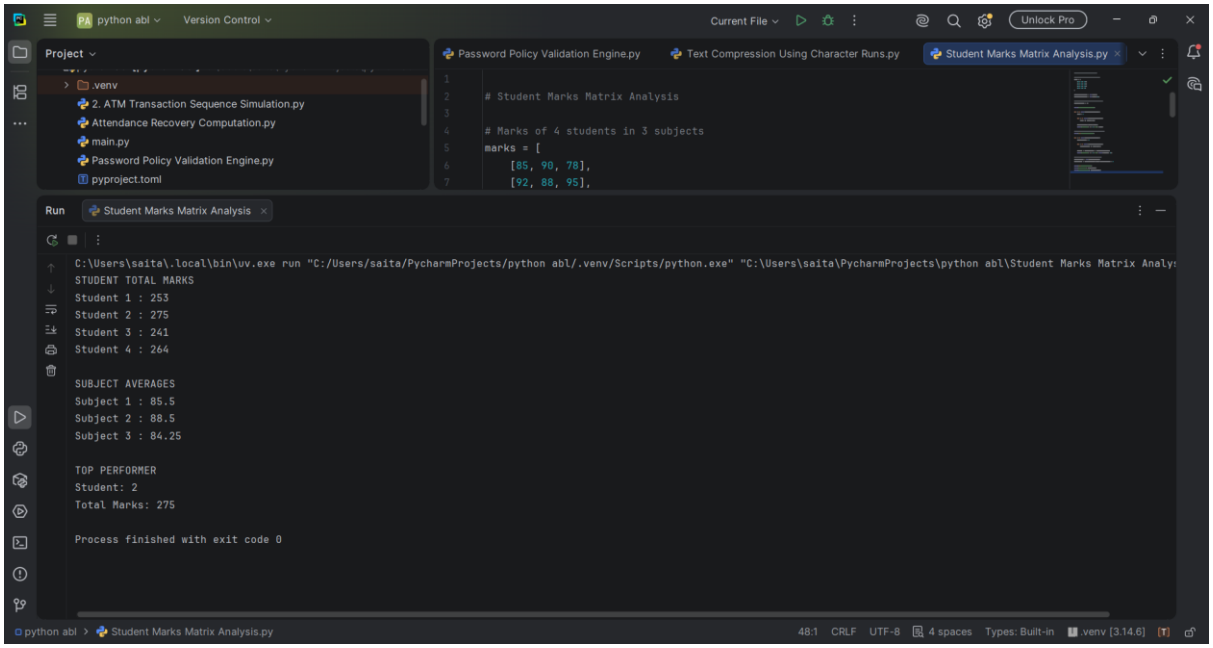
print("\nTOP PERFORMER")
print("Student:", top_student)
print("Total Marks:", highest_total)
```

8. Explanation of the Code

1. Creating the marks matrix: The nested list marks stores marks of four students in three subjects. Each inner list contains the marks of one student.
2. Counting students and subjects: The len() function determines the number of students and subjects in the matrix.
3. Calculating student totals: The outer for loop selects each student. The inner loop adds the marks of all subjects and stores the result in student_totals.
4. Calculating subject averages: The outer loop selects each subject, while the inner loop adds the marks obtained by all students in that subject. The total is divided by the number of students.

5. Rounding averages: The round(average, 2) function displays the average up to two decimal places.
6. Finding the top performer: The max() function finds the highest total marks, and index() identifies the first student with that total. Adding 1 converts the zero-based index into the student number.
7. Displaying results: The print() statements display each student's total, subject averages, and the top-performing student's details.

9. Output



10. Conclusion

The Student Marks Matrix Analysis program successfully analyzes marks stored in a two-dimensional list. It calculates the total marks of each student, determines the average marks for each subject, and identifies the top-performing student. This program demonstrates the practical use of nested lists, nested loops, arithmetic operations, and conditional logic in Python. It also helps students understand how Python can be used to process and analyze academic data efficiently.